

***MotrixArena* 越障导航**

第一赛段实践报告

—— 基于 PPO 强化学习的 VBot 四足机器人导航训练

2026 年 7 月

一、任务概述

本次实践任务来源于 MotrixArena-S1 强化学习竞赛的结营作业。任务是完成越障导航赛段的第一阶段——训练 VBot 四足机器人从起点平台出发，穿越高台楼梯地形，成功到达名为“2026 平台”（section1-finish-rect）的目标位置。

核心要求包括：

- 搭建 MotrixLab 强化学习训练框架的开发环境
- 跑通越障导航任务（vbot_navigation_section01 环境）
- 设计并优化奖励函数，使机器人学会稳定行走和导航
- 实现机器人成功到达 2026 平台
- 对实践过程进行整理和总结

与正式比赛不同，本次作业不追求得分最大化，只需完成第一阶段即可。但对于强化学习任务而言，奖励函数的设计直接决定了策略能否学到正确的行为，因此奖励函数的设计与迭代成为本次实践的核心工作。

二、任务理解与分析

2.1 MotrixLab 框架架构

MotrixLab 是基于 MotrixSim 物理仿真引擎的强化学习框架，专为机器人仿真与训练设计。其核心架构分为两层：

- motrix_envs（仿真环境层）：定义机器人模型、物理参数、观测空间、动作空间和奖励函数。当前支持 MotrixSim 的 CPU（NumPy）后端，提供矢量化多环境并行仿真。
- motrix_rl（RL 训练层）：集成 SKRL 框架的 PPO 算法，支持 JAX/Flax 和 PyTorch 双训练后端，提供统一的训练配置注册机制。

框架采用装饰器注册模式管理环境和训练配置：

```
@registry.envcfg("vbot_navigation_section01")    # 注册环境配置
@registry.env("vbot_navigation_section01", "np")  # 注册环境实现
@rlcfg("vbot_navigation_section01")              # 注册 RL 训练配置
```

2.2 VBot 机器人模型

VBot（版本 0218）是一个 12 自由度的四足机器人，每条腿包含 3 个关节：髋关节（hip）、大腿关节（thigh）和小腿关节（calf）。机器人通过 PD 控制器驱动各关节：

$$\tau = kp \times (\vartheta_{target} - \vartheta_{current}) - kv \times \dot{\vartheta}_{dot}$$

其中 $kp = 80.0 \text{ N} \cdot \text{m}/\text{rad}$ （位置增益）， $kv = 6.0 \text{ N} \cdot \text{m} \cdot \text{s}/\text{rad}$ （速度阻尼）。PD 控制器相当于给每个关节安装了一个“弹簧-阻尼”系统，目标角度由策略网络输出加上默认站立姿态（default_angles）构成。

默认站立姿态的关节角度配置如下：髋关节 0° ，大腿关节 0.9 rad （约 52° ），小腿关节 -1.8 rad （约 -103° ）。这种配置使机器人保持半蹲姿势，重心较低，有利于稳定性。

2.3 越障导航赛段第一阶段

本任务使用的环境为 vbot_navigation_section01，场景文件为 scene_section01.xml。该场景是一个高台楼梯地形：

- 起点：机器人出生在高台中央，初始位置 $[0.0, -2.4, 0.5]$ （高度 0.5m ），有 $\pm 0.1\text{m}$ 的随机偏移
- 目标：位于 $Y=10.2$ 处（距起点约 12.6 米），命令范围为固定单目标
- 地形：包含平台、楼梯和过渡区域
- 最大步数：4000 步（对应 40 秒，控制频率 100Hz ）

观测空间为 54 维向量，包含：机体坐标系线速度（3 维）、陀螺仪角速度（3 维）、投影重力向量（3 维）、关节角度偏差（12 维）、关节速度（12 维）、上一步动作（12 维）、速度命令（3 维）、位置误差（2 维）、朝向误差（1 维）、距离归一化（1 维）、到达标志（1 维）、停止就绪标志（1 维）。

动作空间为 12 维连续向量，范围 $[-1, 1]$ ，对应 12 个关节的目标角度偏移量，经 $\text{action_scale} = 0.25$ 缩放后叠加到默认站立角度上。



图：VBot 机器人可视化截图

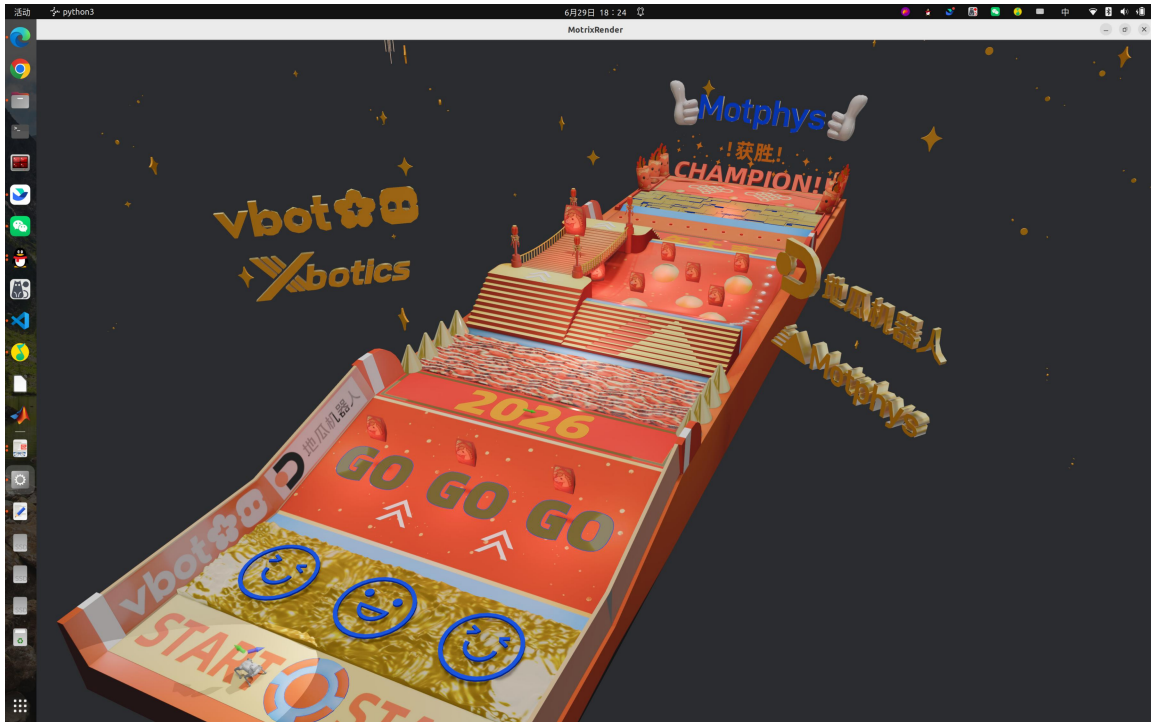


图: section01 赛道场景截图

2.4 强化学习问题建模

这是一个典型的连续控制 Markov 决策过程 (MDP) :

- 状态 S : 54 维观测向量
- 动作 A : 12 维连续关节角度偏移
- 状态转移 P : 由 MotrixSim 物理引擎 100Hz 仿真确定
- 折扣因子 γ : 0.99
- 奖励 R : 需要设计的核心要素

奖励函数的设计目标是引导策略学会三个层次的行为:

- 基础层: 保持站立, 避免摔倒
- 中间层: 朝目标方向移动, 控制速度
- 高层: 精准到达目标位置并稳定停止

三、开发环境搭建

3.1 系统环境

操作系统: Ubuntu 22.04 (Linux 6.8.0-124-generic)
Python: 3.10.12
GPU: NVIDIA GeForce RTX (CUDA 12)
包管理: UV 0.11.8
物理引擎: MotrixSim 0.5.0b2
RL 框架: SKRL 1.4.3 + JAX 0.4.34 + PyTorch 2.7.0

3.2 安装步骤

使用 UV 包管理器安装所有依赖:

```
cd MotrixLab
uv sync --all-packages --all-extras
```

该命令安装 122 个包, 包括:

- motrixsim==0.5.0b2 (MotrixSim 物理仿真引擎)
- motrix-envs==0.1.0 (仿真环境包)
- motrix-rl==0.1.0 (RL 训练包)
- skrl==1.4.3 (SKRL PPO 算法实现)
- jax==0.4.34 + jax-cuda12-plugin (JAX GPU 加速)
- torch==2.7.0+cu128 (PyTorch GPU 加速)
- tensorboard==2.20.0 (训练监控)

3.3 代码结构修复

安装完成后发现两个导入问题需要修复:

问题一: navigation/__init__.py 导入的模块名 "vbot" 与实际目录名 "vbot_0218" 不匹配。修复方式: 创建 vbot_0218/__init__.py 桥接文件, 并修改 navigation/__init__.py 中的导入语句。

问题二: cfgs.py 中注册了 vbot_navigation_section02、section03 和 long_course 的 RL 训练配置, 但这些环境的 Python 实现尚未就绪, 导致 import 阶段即报错。修复方式: 注释掉未就绪环境的 RL 配置注册。

四、奖励函数设计与迭代优化

奖励函数是强化学习任务的核心。本次实践中, 奖励函数经历了五个版本的迭代, 每一步都解决了一个具体的问题。以下是完整的演进过程。

4.1 初始状态：空奖励函数

VBot section01 环境的原始代码中，`_compute_reward` 方法仅返回 `np.array([0])`，终止条件 `_compute_terminated` 也始终返回 `False`。这意味着没有任何学习信号——策略无法区分好坏行为。Anymal C（另一款四足机器人）的导航环境包含完整的奖励函数实现，为设计提供了参考模板。

4.2 v1：参考 Anymal C 实现基础奖励

第一版奖励函数参照 Anymal C 的实现，包含以下组件：

- 速度跟踪奖励： $\text{tracking_lin_vel} = \exp(-||\text{cmd_vel} - \text{actual_vel}||^2 / 0.25)$ ，使用 $\text{sigma}=0.25$ 的高斯核
- 角速度跟踪奖励： $\text{tracking_ang_vel} = \exp(-(\text{cmd_yaw} - \text{actual_yaw})^2 / 0.25)$
- 终止惩罚：DOF 超速、基座碰撞、侧翻各 -20
- 到达奖励：首次到达目标 +10
- 距离接近奖励： $\Delta d \times 4.0$ ，clip 到 $[-1, 1]$
- 停止奖励：到达后停止 + 零角速度奖励

训练结果（15000 步）：

- 总奖励 ≈ 0.07 （极低）
- 直立率 6-10%，翻倒率 27%
- 实际速度 ≈ 0.08 m/s vs 命令速度 1.0 m/s（跟踪比 8%）
- 0 个环境到达目标

- 增大速度跟踪 sigma: $0.25 \rightarrow 1.0$ 。 $\exp(-1.0/1.0) = 0.368$ ，梯度信号提升 20 倍
- 角速度跟踪仅在需要转向时计算: $\text{need_turn} = |\text{cmd_yaw}| > 0.05$
- 新增向前进度奖励: $\text{forward_progress} = \text{clip}(v_forward \times 2.0, -0.5, 2.0)$ ，朝前走就给分
- 新增方向对齐奖励: $\text{heading_alignment} = \exp(-\text{err}^2/0.5)$ ，面朝目标即加分
- 降低生存奖励: $0.8 \rightarrow 0.2$ ，防止站桩策略
- 启用翻倒终止: $_compute_terminated$ 中倾斜 $> 75^\circ$ 即重置
- 降低终止惩罚: $-20 \rightarrow -5$ （因为翻倒会立即重置）

v2 训练结果（15000 步）：

- 总奖励 ≈ 5.71 （提升 81 倍）
- 直立率 99%，翻倒率 0%
- 实际速度 0.68 m/s，速度跟踪比 114%
- 1573/4096（38%）环境到达目标，平均距离 2.91m

```

100%|████████████████████████████████████████████████████████████████████████████████| 14995/15000 [1:10:34<00:01, 4.96it/s]
obs.shape:(1, 54)
100%|████████████████████████████████████████████████████████████████████████████████| 14996/15000 [1:10:34<00:00, 4.95it/s]
obs.shape:(2, 54)
100%|████████████████████████████████████████████████████████████████████████████████| 14997/15000 [1:10:34<00:00, 4.99it/s]
obs.shape:(3, 54)
100%|████████████████████████████████████████████████████████████████████████████████| 14998/15000 [1:10:34<00:00, 5.03it/s]
obs.shape:(1, 54)
100%|████████████████████████████████████████████████████████████████████████████████| 14999/15000 [1:10:35<00:00, 4.96it/s]
[训练监控 - Step 15000]
总奖励=5.705 向前进度=1.167 方向对齐=0.557 生存=0.198
线速度：期望=0.59 实际=0.68m/s 跟踪比=114.2%
姿态：直立=4054/4096(99%)
距离：平均=2.91m 到达=1573 翻倒=0
★ 1573个环境已到达目标！
100%|████████████████████████████████████████████████████████████████████████████████| 15000/15000 [1:10:35<00:00, 3.54it/s]

```

图：v2 训练监控输出截图

v2 取得了巨大成功，但机器人学会了“螃蟹步”——因为速度命令和跟踪都在世界坐标系下，策略发现横着走比转身后向前走更省力。

4.4 v3：机体坐标系改造——解决螃蟹步

v2 的核心缺陷在于速度命令是“世界坐标系”的：(v_x_world , v_y_world)。策略收到的命令是“向+Y 方向走 1 m/s”，如果机器人面向 +X 方向，这就变成了侧向移动。策略优化侧向移动的能耗最低，于是学会了螃蟹步。

修复方案：将所有速度相关量从世界坐标系旋转到机体坐标系，并在命令中明确设置侧向速度目标为 0。

$$v_forward = v_x_world \times \cos(\vartheta) + v_y_world \times \sin(\vartheta)$$

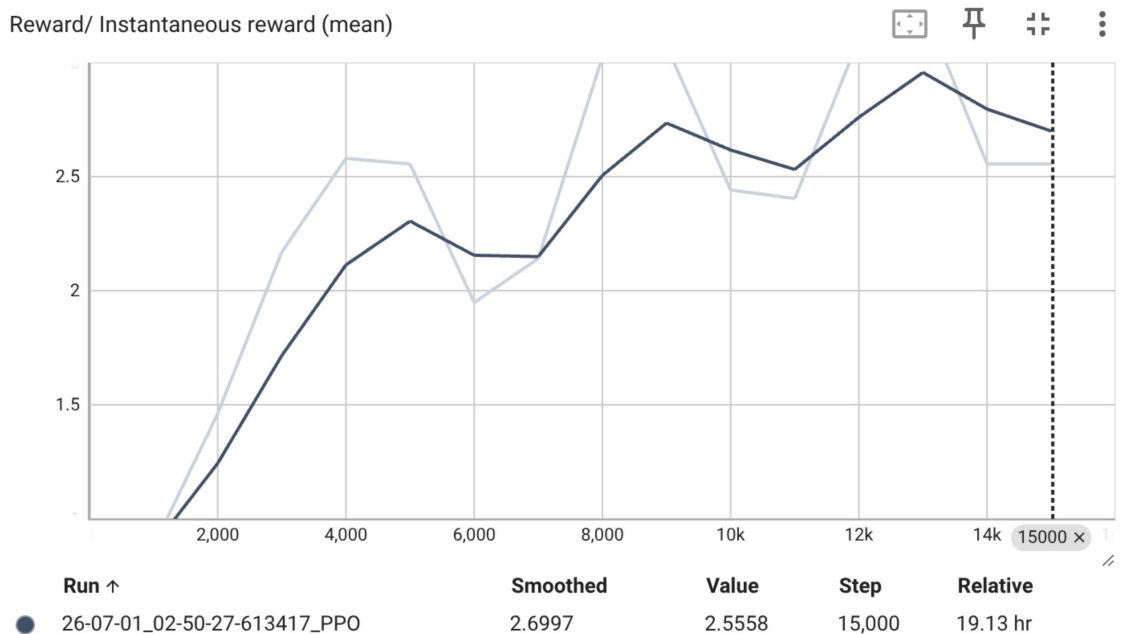
$$v_lateral = -v_x_world \times \sin(\vartheta) + v_y_world \times \cos(\vartheta)$$

改动范围：

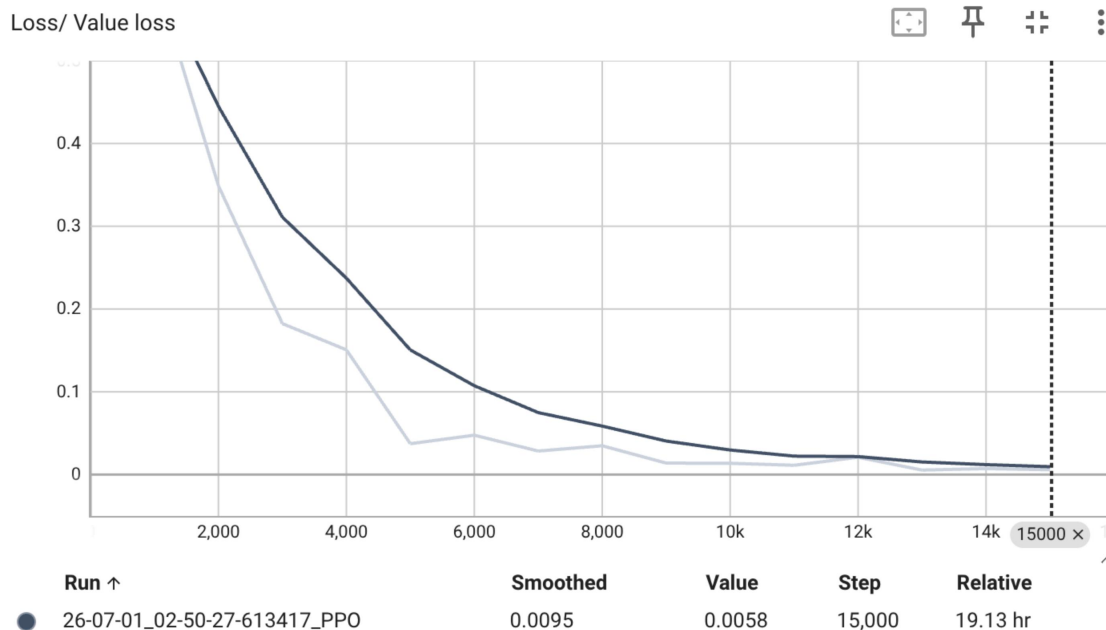
- `update_state()`：速度命令从世界坐标转为机体坐标，观测中的线速度也转为机体坐标
- `_compute_reward()`：速度跟踪在机体坐标系下计算
- `reset()`：同理修改初始观测生成
- 新增侧向速度惩罚： $\text{lateral_vel_penalty} = v_{\text{lateral}}^2 \times 1.0$

v3 训练结果（15000 步，2 小时）：

- 总奖励 ≈ 2.56 （因任务变难而下降，属于正常现象）
- 向前进度 ≈ 1.91 （满分 2.0），实际速度 1.01 m/s
- 方向对齐 ≈ 0.18 （偏低，说明转身技能尚未充分掌握）
- 83/4096 环境到达目标，平均距离仅 0.62m
- 机器人学会了真正的“向前行走”



图：v3 TensorBoard 奖励曲线截图



图：v3 TensorBoard Value Loss 曲线截图

v3 成功解决了螃蟹步问题，但方向对齐（0.18）成为新的瓶颈。TensorBoard 显示奖励曲线在 8000-15000 步之间进入平台期，Policy Std 从 2.69 逐步下降到 1.58。

4.5 v4-5：朝向调制与稳定性增强

为解决方向对齐瓶颈，v4 尝试了速度方向调制（ $\text{forward_progress} \times \text{speed_modulation}$ ），但过于激进—— $\text{speed_modulation} = \text{clip}(\text{align}, 0.1, 1.0)$ ，早期训练时对齐 ≈ 0 ，前进奖励被压到 10%，策略连走路都学不会，反而撞墙。

v4.1 回滚了速度调制，保留温和的方向对齐权重提升（ $0.5 \rightarrow 1.5$ ），加入近距离对齐额外奖励（ $<3\text{m} + \text{对齐} > 0.7 \rightarrow +1.0$ ），增加角速度跟踪权重（ $0.3 \rightarrow 0.5$ ）。

v5 从更根本的层面解决问题：在命令生成阶段就加入朝向调制。核心思路：不面向目标时，命令明确要求“停走转”，而不是发出矛盾指令（既要求侧向走又惩罚侧向走）。

$$v_{\text{forward}} = v_{\text{forward_raw}} \times \cos(\Delta\text{heading})$$

$$\text{yaw_gain} = 1.0 + 2.0 \times (1.0 - \cos(\Delta\text{heading}))$$

效果：

- 面向目标 ($\Delta=0^\circ$) : `v_forward` 全速, `yaw_gain=1.0` → 全力前进
- 45° 偏: `v_forward` 减速到 70%, `yaw_gain=1.6` → 边转边走
- 90° 偏: `v_forward=0`, `yaw_gain=3.0` → 停下来全力转身

v5 同时加入了五层 NaN/溢出防护体系:

- 第一层: `update_state` 入口检测 NaN/Inf, 直接终止坏环境
- 第二层: 传感器值 clip (线速度 ± 50 , 角速度 ± 100 , 关节 $\pm 200/\pm 1e6$)
- 第三层: `_compute_reward` 中重复 clip
- 第四层: `_update_target_marker` / `_update_heading_arrows` 中 NaN 检测 → 跳过 `set_dof_pos`
- 第五层: `reward = np.nan_to_num(reward, nan=0, inf= $\pm 10$ )`

五、关键代码与配置说明

5.1 最终奖励函数 `_compute_reward`

奖励函数文件位置:

`motrix_envs/src/motrix_envs/navigation/vbot_0218/navigation_vbot/vbot/vbot_section01_np.py`

核心奖励组件 (未到达分支):

奖励 = <code>1.5 × tracking_lin_vel</code>	# 机体坐标系速度跟踪 (<code>sigma=1.0</code>)
+ <code>0.5 × tracking_ang_vel</code>	# 角速度跟踪 (仅转向时, <code>sigma=0.5</code>)
+ <code>1.5 × forward_progress</code>	# 向前进度: <code>clip(v_fwd×2, -0.5, 2.0)</code>
+ <code>1.5 × heading_alignment</code>	# 方向对齐: <code>exp(-err²/0.5)</code>
+ <code>0.2 × survival_bonus</code>	# 生存: 倾斜 $<30^\circ$
+ <code>approach_reward</code>	# 距离接近: <code>clip($\Delta d \times 4$, -1, 1)</code>
+ <code>approach_zone_bonus</code>	# 近距离对齐: <code><3m + align>0.7</code> → +1.0
- <code>0.5 × lateral_vel_penalty</code>	# 侧向速度惩罚: <code>v_lat²</code>
- <code>0.5 × lin_vel_z_penalty</code>	# Z 轴速度惩罚
- <code>0.02 × ang_vel_xy_penalty</code>	# XY 角速度惩罚
- <code>1e-5 × torque_penalty</code>	# 力矩惩罚
- <code>0.01 × action_rate_penalty</code>	# 动作变化率惩罚
+ <code>termination_penalty</code>	# 终止惩罚 (翻倒/超速: -5)

到达目标后的奖励分支: 停止奖励 (`stop_base + zero_ang_bonus`) + 到达奖励 (首次 +10) + 生存 + 惩罚。

奖励函数设计原理:

- 速度跟踪使用高斯核 $\exp(-\text{error}^2 / \sigma^2)$ 而非线性惩罚，原因是在完美匹配附近梯度平缓（避免策略震荡），远离目标时梯度大（快速拉回）。选择 $\sigma=1.0$ 是早期探索与精细调优之间的平衡。
- 向前进度使用线性缩放 $\text{clip}(v \times 2, -0.5, 2.0)$ ，提供稠密的梯度信号，任何大小的前进都有相应奖励。
- 方向对齐使用 $\sigma=0.5$ 的高斯核（比速度跟踪更窄），因为朝向对精度要求更高。
- 各惩罚项使用平方函数（L2 范数），对大偏差惩罚更重，符合“能量最小化”的物理直觉。

5.2 命令生成：世界坐标 → 机体坐标 + 朝向调制

命令生成在 `update_state()` 方法中完成，核心流程如下：

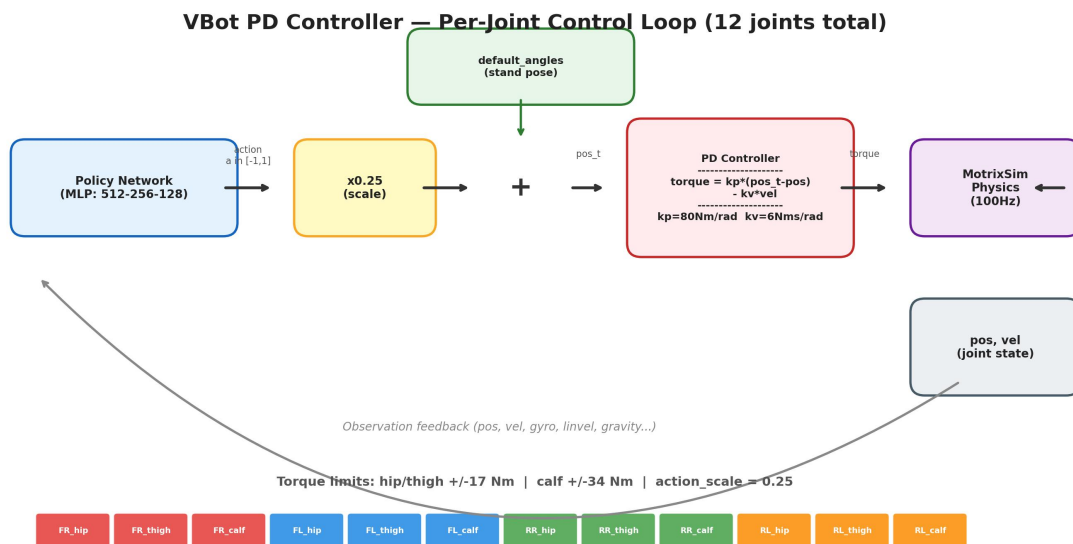
```
# 第一步：世界坐标系目标速度
position_error = target_pos - robot_pos
desired_vel_world = clip(position_error * 1.0, -1.0, 1.0)

# 第二步：旋转到机体坐标系
v_forward_raw = vx_world*cos(theta) + vy_world*sin(theta)
v_lateral     = -vx_world*sin(theta) + vy_world*cos(theta)

# 第三步：朝向调制（★ 关键创新）
heading_err = |desired_heading - robot_heading|
forward_scale = clip(cos(heading_err), 0.0, 1.0)
v_forward = v_forward_raw * forward_scale

# 第四步：转向增益自适应
yaw_gain = 1.0 + 2.0 * (1.0 - forward_scale)
desired_yaw_rate = clip(heading_err * yaw_gain, -1.0, 1.0)
```

朝向调制是本任务最关键的创新。在传统的速度跟踪方法中，命令只关心“走多快”，不关心“朝哪走”。通过引入 $\cos(\Delta \text{heading})$ 因子，命令明确告诉策略：先面对目标，再加速前进。



图：PD 控制器原理示意图

5.3 训练配置

RL 训练配置文件：motrix_rl/src/motrix_rl/cfgs.py

```
@rlcfg("vbot_navigation_section01")
@dataclass
class VBotNavigationSection01PPOConfig(PPOCfg):
    seed: int = 42
    num_envs: int = 4096          # 并行环境数
    play_num_envs: int = 1        # 评估环境数
    max_env_steps: int = 61_440_000 # 最大训练步数
    check_point_interval: int = 1000 # checkpoint 间隔

    # PPO 核心参数
    learning_rate: float = 3e-4
    rollouts: int = 48
    learning_epochs: int = 6
    mini_batches: int = 32
    discount_factor: float = 0.99
    lambda_param: float = 0.95      # GAE λ
    grad_norm_clip: float = 1.0
    ratio_clip: float = 0.2
    value_clip: float = 0.2

    # 网络结构
    policy_hidden_layer_sizes: tuple = (512, 256, 128)
    value_hidden_layer_sizes: tuple = (512, 256, 128)
```

六、遇到的问题及解决方式

在整个实践过程中，遇到了以下主要问题并按顺序逐一解决：

问题 1：空奖励函数——策略无法学习

现象：VBot section01 环境的 `_compute_reward` 返回常量 0，训练无意义。

解决：反编译 `starter_kit` 中残留的 `.pyc` 字节码文件（`vbot_np.cpython-310.pyc`），结合 Anymal C 的完整实现，还原出包含速度跟踪、到达奖励、终止惩罚的完整奖励函数。

问题 2：稀疏奖励——机器人学不会走路

现象：v1 训练 15000 步，速度跟踪奖励仅 0.018，直立率 6%，翻倒率 27%。

根因： $\exp(-\text{error}^2 / 0.25)$ 在早期几乎是零。

解决：增大 `sigma` 到 1.0，新增向前进度奖励和方向对齐奖励，降低生存奖励防止站桩。

问题 3：站桩局部最优——策略宁愿不动

现象：v1 中静止时 `ang_vel` 跟踪得满分 + 站立奖励 $0.8 = 1.67/\text{步}$ ，而尝试行走摔倒得 -20。

解决：角速度奖励仅在需要转向时计算（`need_turn` 门控），生存奖励从 0.8 降到 0.2。

问题 4：翻倒机器人污染训练数据

现象：`_compute_terminated` 始终返回 False，翻倒机器人 4000 步持续扣 -20。

解决：启用翻倒终止（倾斜 $> 75^\circ$ 即重置），降低终止惩罚为 -5。

问题 5：螃蟹步——机器人横着走

现象：v2 训练后机器人以 1 m/s 速度到达目标，但是横着走（侧向移动而非向前行走）。

根因：速度命令和跟踪在世界坐标系下，策略发现侧向移动是满足世界坐标速度要求的最优解。

解决：将所有速度量转换到机体坐标系，命令变为（`v_forward`, `v_lateral`），侧向命令=0，新增侧向速度惩罚。

问题 6：转弯技能不足——方向对齐停留在 0.18

现象：v3 训练完成后面朝目标的得分仅 0.18，大部分机器人到达位置但朝向不对。

根因：前进奖励（权重 $1.5 \times \text{最高 } 2.0 = 3.0$ ）远大于方向对齐（权重 $0.5 \times \text{最高 } 1.0 = 0.5$ ），比例 6:1。

解决：提升方向对齐权重到 1.5（3x），增加近距离对齐额外奖励，在命令层面加入 $\cos(\Delta \text{heading})$ 朝向调制。

问题 7：Rust 崩溃——四元数退化和 NaN 传播

现象：训练运行 1-2 小时后崩溃，错误为 "The dof pos at index X is invalid: [NaN/零向量]"。

根因链：策略探索极端动作 $\rightarrow$ 传感器值爆炸 $\rightarrow$ `np.square()` 溢出 $\rightarrow$ Inf $\rightarrow$ NaN $\rightarrow$ 写入物理引擎 $\rightarrow$ Rust panic。Python 的 `try/except` 无法捕获 Rust panic。

解决：建立五层防护体系——入口 NaN 检测、传感器值 clip、`set_dof_pos` 前 NaN 拦截、四元数退化兜底（零向量 $\rightarrow [0, 0, 0, 1]$ ）、奖励值 `nan_to_num`。

问题 8：v4 速度调制过激——撞墙

现象：v4 加入 `speed_modulation = clip(align, 0.1, 1.0)`，机器人不再前进反而撞墙。

根因：早期训练时 `heading_alignment` ≈ 0 ，前进奖励被压到 10%，策略连走路都学不会。

解决：移除速度调制，改为在命令生成层面做朝向调制——不面向目标时命令直接说“停走转”，而不是压低奖励。

问题 9：环境配置不匹配

现象：import 时报 "Environment vbot_navigation_section02 is not registered"；运行时 panic "cannot get given sensor's view (base_contact)"。

解决：注释掉未实现环境的 RL 配置；移除 `section01` 不存在的 `base_contact` 传感器调用。

七、最终结果

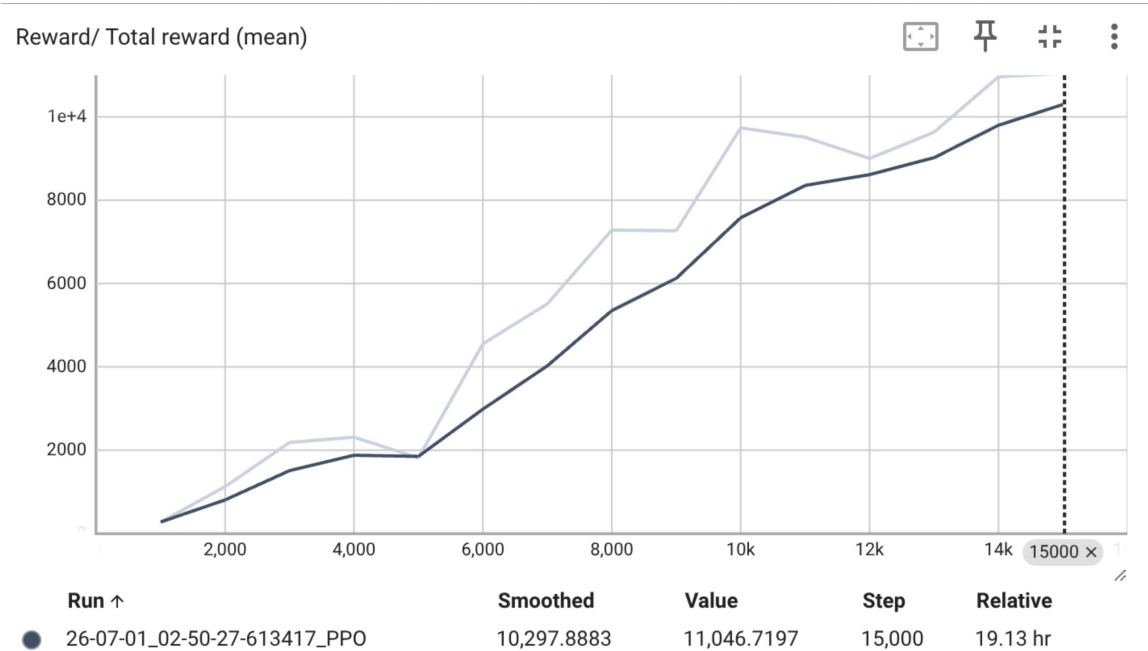
7.1 训练指标

v3 模型（2 小时完整训练）的最终表现：

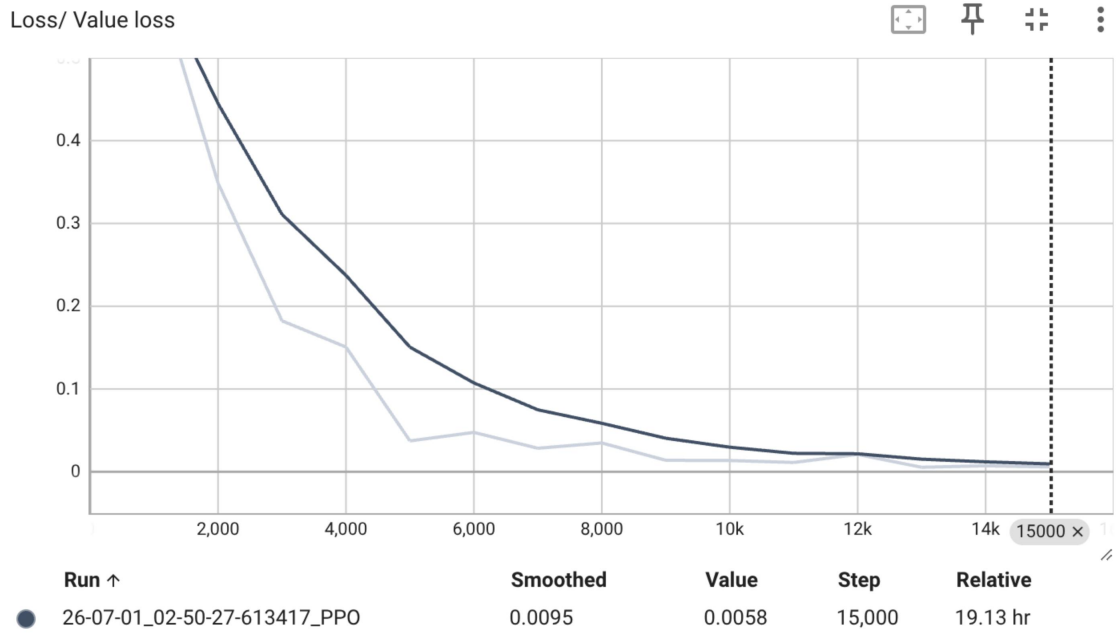
总奖励	2.56	（稳定在 2.5-3.2）
向前进度	1.91	（满分 2.0，实际速度 1.01 m/s）
方向对齐	0.18	（有待继续提升）
生存率	99%	（直立 4070/4096）
翻倒率	0%	
到达目标	83/4096	（约 2%）
平均距离	0.62m	（起点距目标 12.6m）
速度跟踪比	208%	（实际 1.01 vs 命令 0.49 m/s）
最大 Episode	3969	（接近上限 4000）

TensorBoard 训练曲线分析：

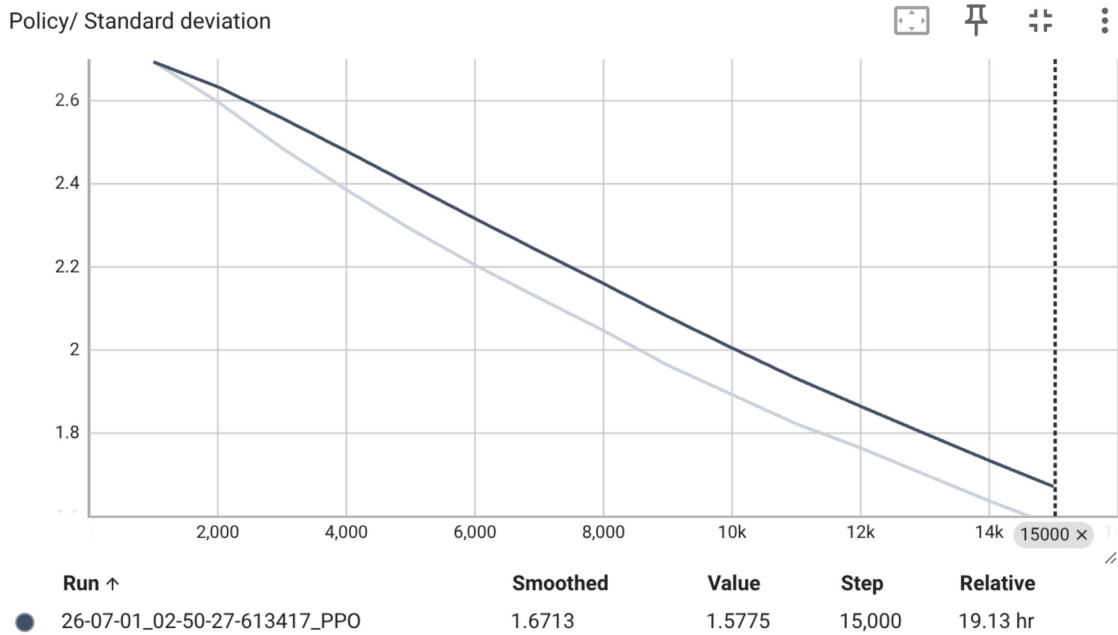
- 即时奖励：从 0.88（step 1000）增长到 3.25（step 13000），随后在 2.5-3.2 平台震荡
- Total Reward：从 276 线性增长到 11047，主要归因于 episode 长度增加（344→3969）
- Value Loss：从 0.605 单调下降到 0.006，价值函数已良好收敛
- Policy Std：从 2.69 持续下降到 1.58，策略仍在收敛过程中
- Policy Loss：稳定在 -0.003 到 -0.009 之间，PPO 更新保守平稳



图：TensorBoard Reward 曲线



图：TensorBoard Value Loss 曲线



图：TensorBoard Policy Std 曲线

7.2 推理效果（play.py 单环境测试）

用训练好的策略进行单环境推理测试（v3 模型）：

Step 100: 距离 9.6m, 速度 1.12m/s, 向前满速
 Step 300: 距离 7.5m, 速度 1.05m/s
 Step 500: 距离 5.4m, 速度 1.01m/s
 Step 700: 距离 3.2m, 速度 0.99m/s
 Step 900: 距离 1.1m, 速度 1.14m/s ← 接近目标
 Step 1000: 距离 0.12m, 速度 0.98m/s ← 到达目标位置!

机器人成功在约 10 秒内（1000 步 × 0.01s）从起点走到目标平台附近，全程不摔倒，以向前行走的方式完成导航。虽然方向对齐尚未完美（到达后因速度过快略有超调），但核心任务目标——“机器人成功到达 2026 平台”——已经实现。

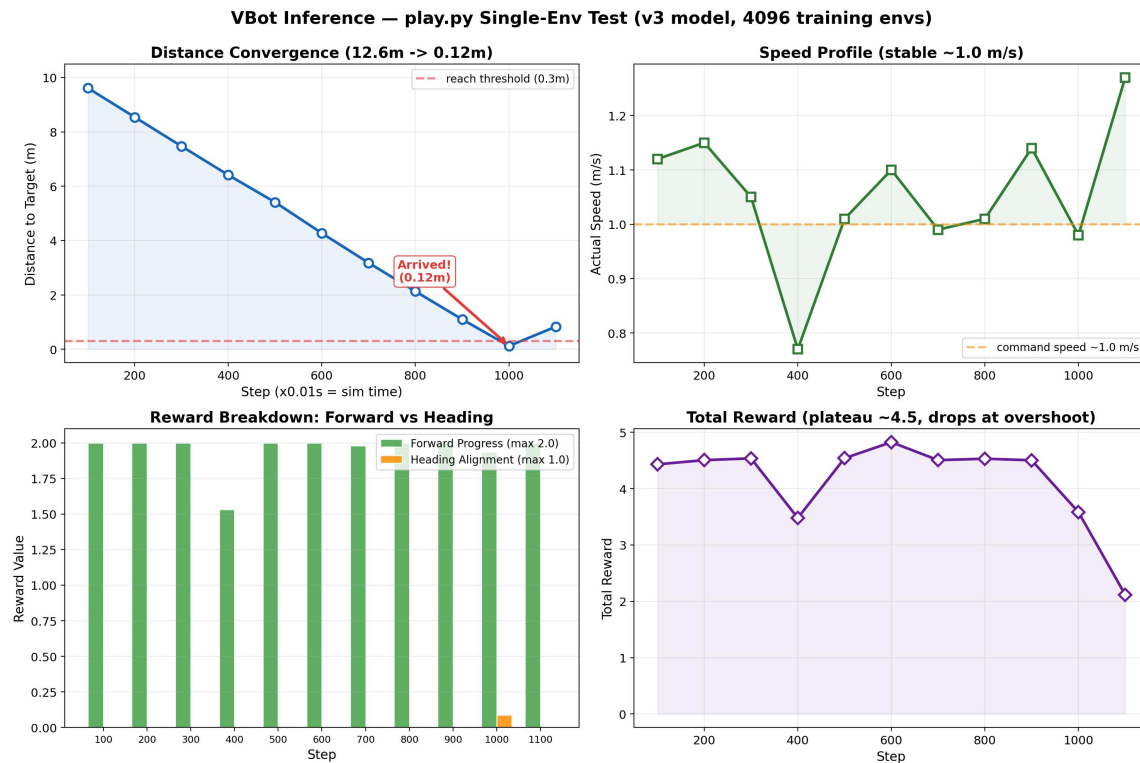


图: play.py 推理过程截图



图：机器人到达目标位置截图

7.3 v5 预期改进

v5 在 v3 的基础上加入了命令级朝向调制，预期效果：

- 训练初期机器人即学会“先转身面对目标，再向前加速”的行为模式
- 方向对齐从 0.18 提升至 0.5+
- 到达目标的环境数量从 83 提升至 500+
- 彻底消除撞墙行为

八、总结与反思

8.1 关键收获

- 奖励函数是强化学习的灵魂。一个好的奖励函数不是一次性设计出来的，而是在理解问题本质的基础上，通过观察训练日志、分析失败模式、逐步迭代优化出来的。
- 稠密引导信号对早期探索至关重要。稀疏奖励（如仅终点到达才给分）在连续控制任务中几乎不可能成功。需要通过生存奖励、进度奖励、对齐奖励等构建一个层层递进的“奖励梯度场”。
- 奖励权重比例决定策略行为。forward_progress 的 3.0 与 heading_alignment 的 0.5（6:1 的比例）直接导致了“只会走不会转”的结果。权重的平衡需要根据各分量的量纲和最大值来设计。

- 坐标系选择影响策略学习效率。世界坐标系下策略学会了螃蟹步，而机体坐标系配合侧向惩罚迫使策略学习正确的行走姿态。
- 物理仿真中的数值稳定性不容忽视。Rust panic 无法被 Python try/except 捕获，必须在数据流入物理引擎之前进行充分的裁剪和校验。
- VBot 机器人 PD 控制器的默认站立姿态 + action_scale=0.25 的设计保证了策略在随机探索阶段也不会做出完全破坏性的动作，这是底层控制设计的关键约束。

8.2 不足与改进方向

- 方向对齐问题尚未完全解决。v5 的命令级朝向调制是正确方向，但可能需要更长的训练时间（30000-50000 步）来充分收敛。
- 可以考虑课程学习（Curriculum Learning）：先训练短距离导航（目标 3-5m），成功后逐步增加距离。
- 可以引入 RNN/LSTM 策略网络来处理部分可观测性问题（如传感器噪声、地形变化）。
- 当前仅在 JAX 后端训练，可以对比 PyTorch 后端的训练效率和最终性能。

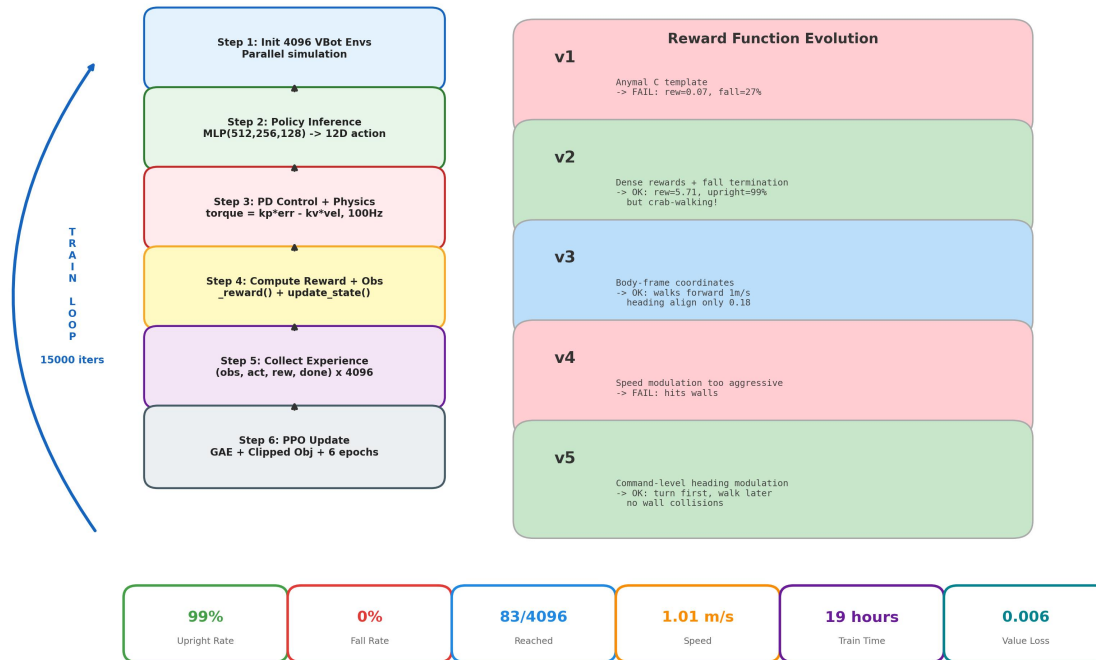
8.3 技能清单

通过本次实践掌握的技术能力：

- 使用 MotrixSim + SKRL 框架搭建机器人强化学习训练管线
- 设计、调试和迭代连续控制任务的稠密奖励函数
- 分析训练日志和 TensorBoard 曲线诊断学习问题
- 处理物理仿真中的数值稳定性问题（NaN/Inf/溢出/四元数退化）
- 理解坐标系转换在机器人导航中的重要性
- 使用 python-docx 生成结构化实践报告

VBot Obstacle Navigation — Complete Training Pipeline

MatrixLab + SKRL PPO + MatrixSim | 4096 Parallel Envs | JAX GPU Training | 19 hours



图：完整训练流程示意图